Práctica 2:

Aprendizaje automático para la detección de tipos de documentos

ÍNDICE

ÍNDICE 2
Instalación 3
Uso del software 3
Explicación Clasificadores 4
Carga de datos 4
Muestra de métricas 4
C1 6
C2 7
C3 9
C4 10
Comparativa final de los clasificadores 12
Imagen seleccionada por consola 13

Instalación

Los requisitos son tener: Python 3.12, OpenCV 4.11, el fichero scanner.py de la primera entrega (lo hemos actualizado para que detectara la función al llamarlo con otro .py, lo hemos incluido en la carpeta) y una serie de librerías de Python.

Uso del software

Para usar el programa se ejecutará desde el terminar: python doc_classifier.py imagen.jpg.

A lo mejor hay que modificar del código alguna ruta dependiendo de donde estén las imágenes de aprendizaje y test. Esto no debería de pasar si se ejecuta dentro de la carpeta entregada.

En la entrega mandamos doc_classifier.py que será el ejecutable, scanner.py que es la práctica anterior y las fotos necesarias para que haga el aprendizaje y testing.

Explicación Clasificadores

Carga de datos

Estas han sido las funciones que hemos usado para cargar las imágenes y las etiquetas de los clasificadores. En los clasificadores c3 y c4 se usa la función deteccionEsquinas que está en scanner.py de la anterior entrega. Estas funciones recorren las carpetas de cada clase, leen las imágenes, las redimensionan y almacenan sus datos en arrays.

```
def cargar_imagenes_y_etiquetas(directorio):
    X, y = [], []
    for clase, etiqueta in clases_dic.items():
        carpeta = os.path.join(directorio, clase)
        for archivo in os.listdir(carpeta):
            if archivo.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp')):
                ruta = os.path.join(carpeta, archivo)
                img = cv2.imread(ruta)
                if img is not None:
                    img = cv2.resize(img, IMAGE_SIZE)
                    X.append(img.flatten())
                    y.append(etiqueta)
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.int32)

def cargar_imagenes_y_etiquetas_c3(directorio):
    X, y = [], []
    for clase, etiqueta in clases_dic.items():
        carpeta = os.path.join(directorio, clase)
        for archivo in os.listdir(carpeta):
            if archivo.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp')):
                ruta = os.path.join(carpeta, archivo)
                img = deteccionEsquinas(ruta)
                if img is not None:
                    img = cv2.resize(img, IMAGE_SIZE)
                    X.append(img.flatten())
                    y.append(etiqueta)
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.int32)
```

Muestra de métricas

Para evaluar el rendimiento de los clasificadores usamos métricas comunes como:

Accuracy, Precision y Recall.

También generamos la matriz de confusión para visualizar los aciertos y errores por clase.

En cada clasificador existe este bloque que abre una ventana para mostrar una matriz de confusión con los resultados.

```
acc_c1 = metrics.accuracy_score(y_test_c1, y_pred_c1)
prec_c1 = metrics.precision_score(y_test_c1, y_pred_c1, average='weighted')
rec_c1 = metrics.recall_score(y_test_c1, y_pred_c1, average='weighted')
print(f"Accuracy del clasificador C1 (SVM): {acc_c1:.2%}")
print(f"Precision: {prec_c1:.2%}")
print(f"Recall: {rec_c1:.2%}")

cm_c1 = confusion_matrix(y_test_c1, y_pred_c1)
ConfusionMatrixDisplay(confusion_matrix=cm_c1,
display_labels=list(clases_dic.keys())).plot(cmap=plt.cm.Blues)
plt.title("C1 - SVM (RGB)")
plt.show()
```

C1

Tras hacer la carga de los datos, dividimos el dataset en conjunto de entrenamiento y de prueba. Hemos utilizado la función creada que esta anteriormente explicada.

```
X_train_c1, y_train_c1 = cargar_imagenes_y_etiquetas(APRENDIZAJE_DIR)
X_test_c1, y_test_c1 = cargar_imagenes_y_etiquetas(TEST_DIR)
```

Creamos el clasificador SVM con kernel lineal

```
svm_c1 = SVC(kernel='linear')
```

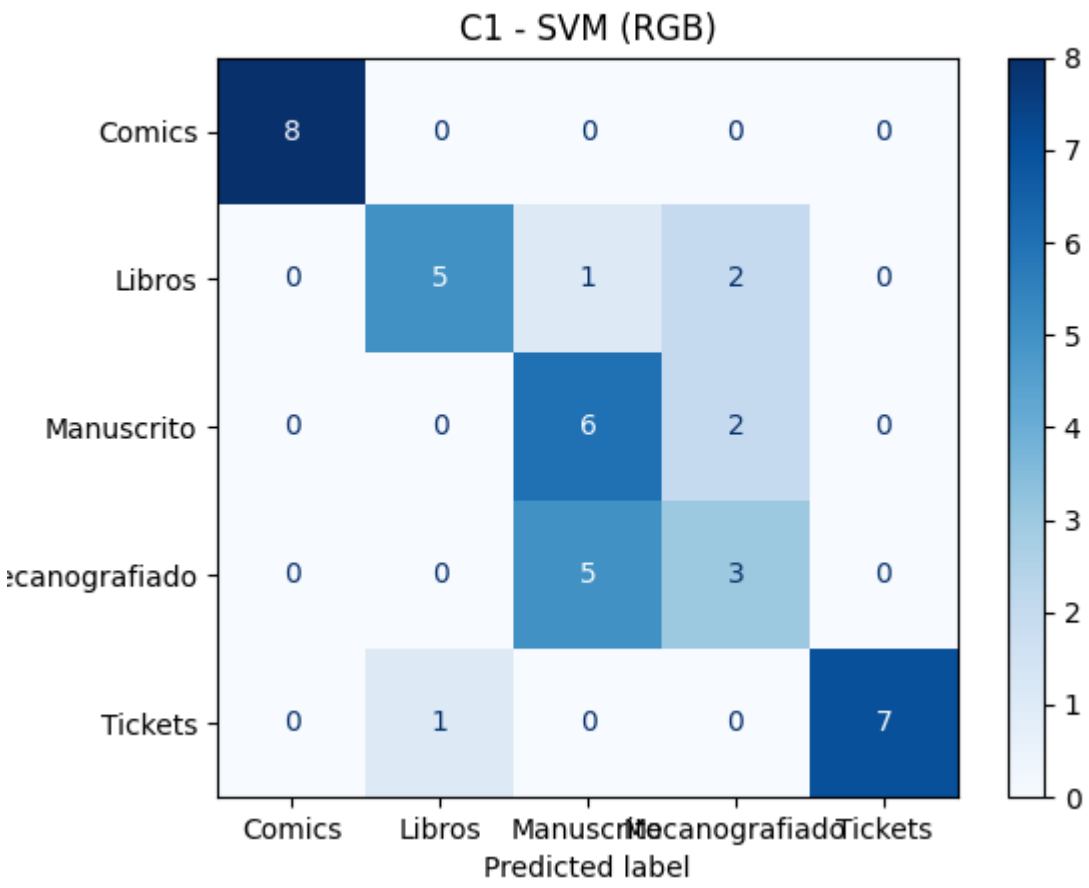
Entrenamos el modelo

```
svm_c1.fit(X_train_c1, y_train_c1)
```

Predecimos los resultados para los patrones de prueba

```
y_pred_c1 = svm_c1.predict(X_test_c1)
```

Los resultados obtenidos, usando el código mostrado en Muestra de datos de métricas, son:



Los resultados printeados por consola:

```
Accuracy del clasificador C1 (SVM): 72.50%
Precision: 75.24%
Recall: 72.50%
```

C2

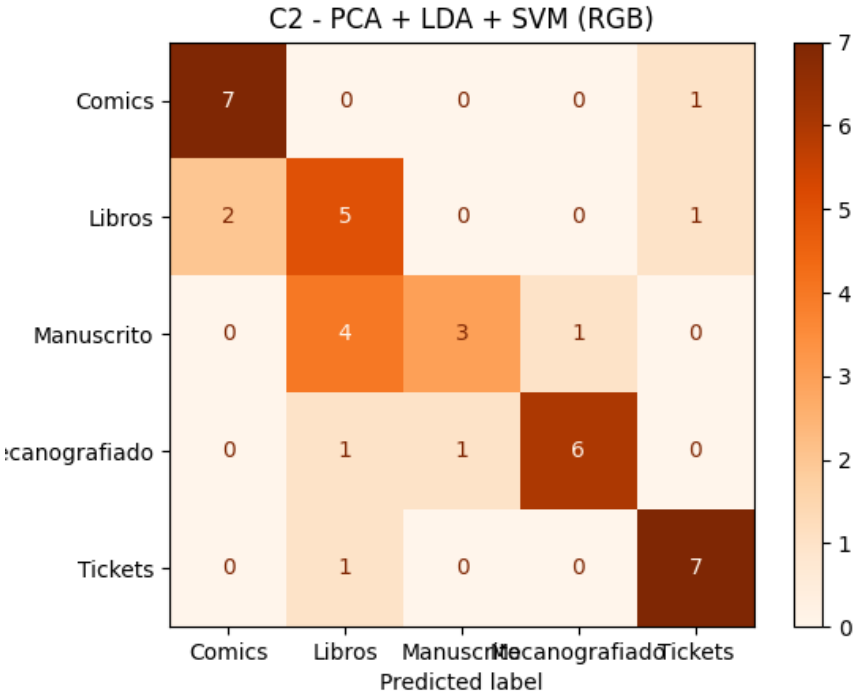
Aplica PCA para reducir la imagen a 30 dimensiones, esto ayuda al algoritmo LDA que crea 4 dimensiones discriminantes, buscando maximizar la separación entre clases y por último se usa SVM con kernel lineal para la clasificación

```
# PCA
pca_c2 = decomposition.PCA(n_components=30)
X_train_pca_c2 = pca_c2.fit_transform(X_train_c1)
X_test_pca_c2 = pca_c2.transform(X_test_c1)

# LDA
lda_c2 = LinearDiscriminantAnalysis(n_components=4)
X_train_lda_c2 = lda_c2.fit_transform(X_train_pca_c2, y_train_c1)
X_test_lda_c2 = lda_c2.transform(X_test_pca_c2)

# SVM
svm_c2 = SVC(kernel='linear')
svm_c2.fit(X_train_lda_c2, y_train_c1)
y_pred_c2 = svm_c2.predict(X_test_lda_c2)
```

Los resultados obtenidos, usando el código mostrado en Muestra de datos de métricas, son:



Los resultados printeados por consola (Estos resultados pueden ir un 65% hasta un 75%)

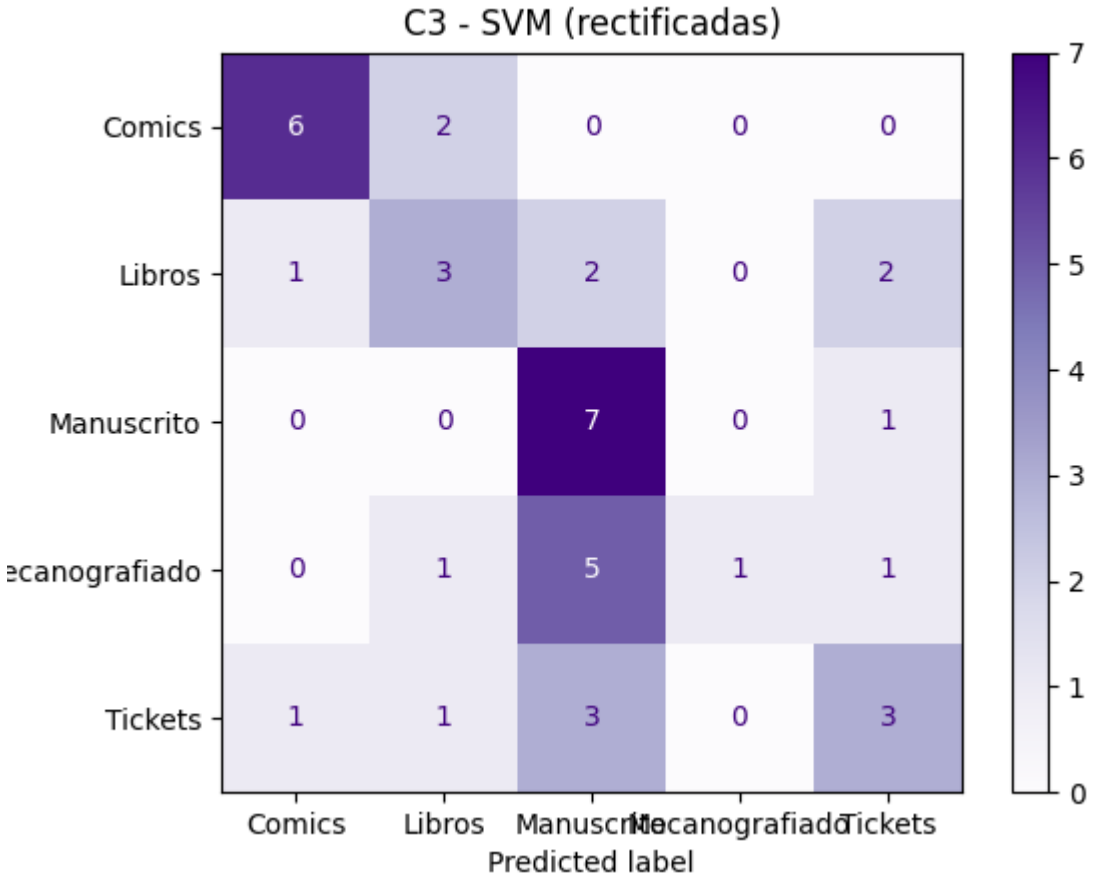
```
C2 Accuracy (PCA+LDA+SVM): 72.50%
Precision: 75.09%
Recall: 72.50%
```

C3

Cargamos las imágenes con las funciones que hemos explicado en Carga de datos usando la función de la práctica anterior y aplicamos SVM como en el C1 para predecir.

```
X_train_c3, y_train_c3 = cargar_imagenes_y_etiquetas_c3(APRENDIZAJE_DIR)
X_test_c3, y_test_c3 = cargar_imagenes_y_etiquetas_c3(TEST_DIR)

svm_c3 = SVC(kernel='linear')
svm_c3.fit(X_train_c3, y_train_c3)
y_pred_c3 = svm_c3.predict(X_test_c3)
```



Los resultados printeados por consola

```
C3 Accuracy (SVM sobre rectificadas): 50.00%
Precision: 60.38%
Recall: 50.00%
```

C4

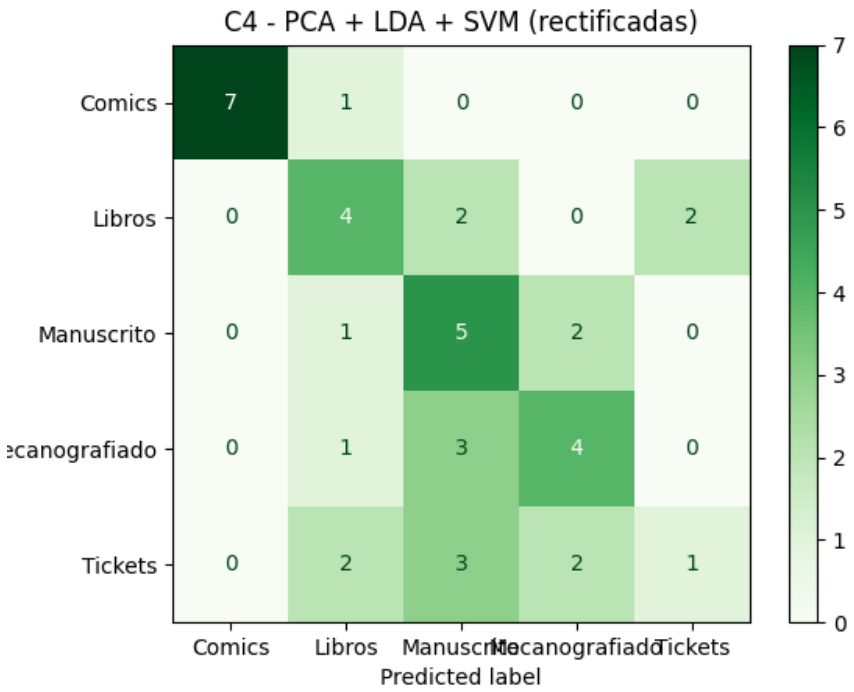
Igual que en el C2 aplicamos las reducciones de dimensiones de PCA a 30 y de LDA en este caso a 2 y predecimos con SVM. A diferencia de en C2 hemos notado que usando LDA con 2 dimensiones discriminantes nos daba un mejor resutados.

```
# PCA
pca_c4 = decomposition.PCA(n_components=30)
X_train_pca_c4 = pca_c4.fit_transform(X_train_c3)
X_test_pca_c4 = pca_c4.transform(X_test_c3)

# LDA
lda_c4 = LinearDiscriminantAnalysis(n_components=2)
X_train_lda_c4 = lda_c4.fit_transform(X_train_pca_c4, y_train_c3)
X_test_lda_c4 = lda_c4.transform(X_test_pca_c4)

# SVM
svm_c4 = SVC(kernel='linear')
svm_c4.fit(X_train_lda_c4, y_train_c3)
y_pred_c4 = svm_c4.predict(X_test_lda_c4)
```

Los resultados obtenidos, usando el código mostrado en Muestra de datos de métricas, son:



Los resultados printeados por consola (Estos resultados pueden ir un 50% hasta un 60%)

```
C4 Accuracy (PCA + LDA + SVM, rectificadas): 60.00%
Precision: 60.20%
Recall: 60.00%
```

Comparativa final de los clasificadores

Estos son los datos obtenidos de todos los clasificadores agrupados:

Clasificador	Accuracy	Precision	Recall
C1	72.50	75.24	72.50
C2	72.50	75.09	72.50
C3	50.00	60.38	50.00
C4	60.00	60.20	60.00

Como podemos observar los clasificadores con más aciertos son C1 y C2. Aunque, C1 es el más estable y, por ende, efectivo.

Según nuestro criterio tienen sentido los resultados:

- C1 es el más efectivo. Esto se debe a que conserva toda la información de las imágenes.
- C2 como reduce las dimensiones y componentes, lo que en algunos casos puede perjudicar la precisión al eliminar características clave y reducir la precisión.
- C3 al usar la práctica anterior para la rectificación. Creemos que como teníamos en la anterior práctica un 70% más o menos de acierto, al fallar en alguna foto puede quitar características clave. Además, los tickets rectificados no tienen su longitud característica.
- C4 arrastra el problema de C3, pero en este caso reducir las dimensiones como en C2 ayuda a encontrar las características debido a que toda el área de la imagen es papel y no se pierde información debido al fondo.

Imagen seleccionada por consola

Utilizamos los modelos ya entrenados para evaluar en cada clasificador

```
foto = sys.argv[1]

# --- Para C1 y C2 ---
img = cv2.imread(foto)
img_rgb = cv2.resize(img, IMAGE_SIZE)
img_flat_rgb = img_rgb.flatten().reshape(1, -1).astype(np.float32)

# C1
pred_c1 = svm_c1.predict(img_flat_rgb)[0]
print(f"\nPredicción C1: {list(clases_dic.keys())[pred_c1]}")

# C2
img_pca = pca_c2.transform(img_flat_rgb)
img_lda = lda_c2.transform(img_pca)
pred_c2 = svm_c2.predict(img_lda)[0]
print(f"Predicción C2: {list(clases_dic.keys())[pred_c2]}")

# --- Para C3 y C4 ---
img_rect = deteccionEsquinas(foto)
img_rect = cv2.resize(img_rect, IMAGE_SIZE)
img_flat_rect = img_rect.flatten().reshape(1, -1).astype(np.float32)

# C3
pred_c3 = svm_c3.predict(img_flat_rect)[0]
print(f"Predicción C3: {list(clases_dic.keys())[pred_c3]}")

# C4
img_pca_r = pca_c4.transform(img_flat_rect)
img_lda_r = lda_c4.transform(img_pca_r)
pred_c4 = svm_c4.predict(img_lda_r)[0]
print(f"Predicción C4: {list(clases_dic.keys())[pred_c4]}")
```

En este caso la imagen que hemos elegido es Libro_1. Nota: se deberá pasar una ruta completa para la imagen.

```
\VA\Pr2VA\Pr2VA\PracticaObligatoria2_2025\MUESTRA\Test\Libros\Libro_1.jpg
```

Y este es el resultado devuelto:

```
Predicción C1: Libros
Predicción C2: Libros
Predicción C3: Libros
Predicción C4: Libros
```